

CS 486/686

Supervised Learning & Generalization

Yuntian Deng

Lecture 17

RN 19.1–19.2, 19.6 · PM 7.1–7.3

Recorded lecture (asynchronous)



This lecture is **pre-recorded** — I'm away at **ICML** this week, so there's no in-person class today. Watch at your own pace.



Due today (Tue Jul 7): Chat 8 and the CS686 project proposal.



Questions? Post on **Piazza** — the TAs are available, and I'll follow up when I'm back.

Search ›

Uncertainty ›

Decisions ›

Learning

Learning goals

- Name the **types of learning** and tell **classification** from **regression**.
- See supervised learning as **fitting parameters to minimize a loss**.
- Derive and fit **linear regression** and **logistic regression**.
- Use **cross-entropy** and **softmax** for classification.
- Explain **overfitting**; control it with **cross-validation** and **regularization**.

Why an agent that learns?



Medical
diagnosis



Spam filtering



Chatbots
(LLMs)



Speech



Image
generation

Learning = improving on future tasks based on experience (data).

- We can't anticipate every situation, and the world keeps changing.
- For most of these tasks we have *no idea how to program a solution* — only how to *show examples*.

Three kinds of learning

Supervised



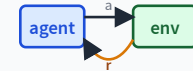
Given inputs + **targets**,
predict targets for new
inputs.
Today's focus.

Unsupervised



No targets — find
structure: clustering,
representations.
L18.

Reinforcement



Learn from rewards —
what to *do*.
L15 (done).

Supervised or unsupervised?

Q1. We have a user's credit-card transactions and want to flag any that look *different* from the rest. We have no fraud labels.

- A. Supervised
- B. **Unsupervised**

B — Unsupervised. No target labels to predict.

Q2. We have historical weather labels (sunny/cloudy/rain/snow) for a date, and we want to predict the same date next year.

- A. **Supervised**
- B. Unsupervised

A — Supervised. Each example has a target (the weather label).

Two flavors of supervised learning

Classification



Target is **discrete** — e.g. dog vs cat.

Q3. Predict next year's weather label (sunny/rain/...) for a date.

Classification — discrete target.

Regression



Target is **continuous** — e.g. tomorrow's temperature.

Q4. Predict next month's price of a house.

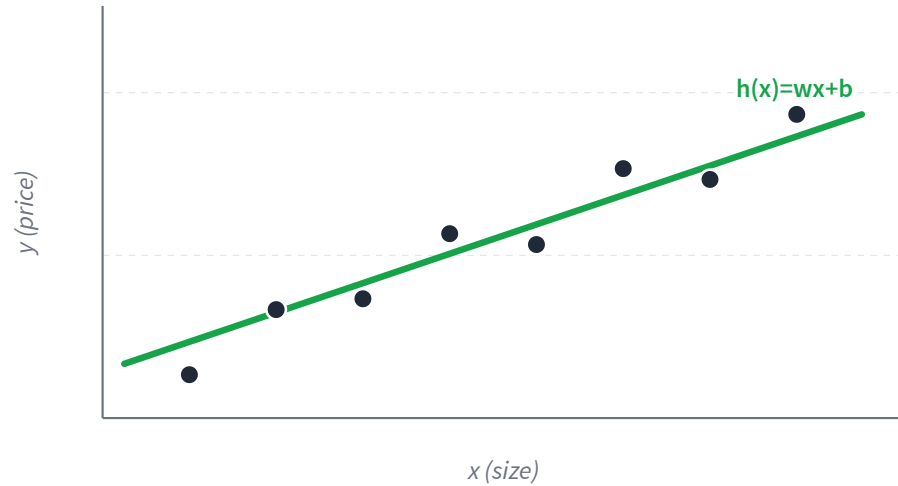
Regression — continuous target.

The recipe behind every supervised learner

1. **Model.** Pick a function $h_{\mathbf{w}}$ with tunable **parameters** $\mathbf{w}$ (weights).
2. **Loss.** Measure how wrong $h_{\mathbf{w}}$ is: $L(\mathbf{w}) = \frac{1}{m} \sum_i \ell(h_{\mathbf{w}}(x_i), y_i)$.
3. **Fit.** Choose $\mathbf{w}$ that makes $L(\mathbf{w})$ small — usually by **gradient descent**.

Change the model and the loss → you get every method in this module (regression → neural nets → transformers).

Linear regression: the model



One real input x , one real output:

$$h_{\mathbf{w}}(x) = w x + b$$

With n features $\mathbf{x} = (x_1, \dots, x_n)$
:

$$h_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$$

Parameters to learn: the slope(s) $\mathbf{w}$ and intercept b .

Linear regression: fitting by least squares

Loss = mean **squared error** between prediction and target:

$$L(\mathbf{w}, b) = \frac{1}{m} \sum_{i=1}^m (h_{\mathbf{w}}(x_i) - y_i)^2$$

Closed form

L is convex — set $\nabla L = 0$ and solve. For features X : $\mathbf{w}^* = (X^\top X)^{-1} X^\top \mathbf{y}$.

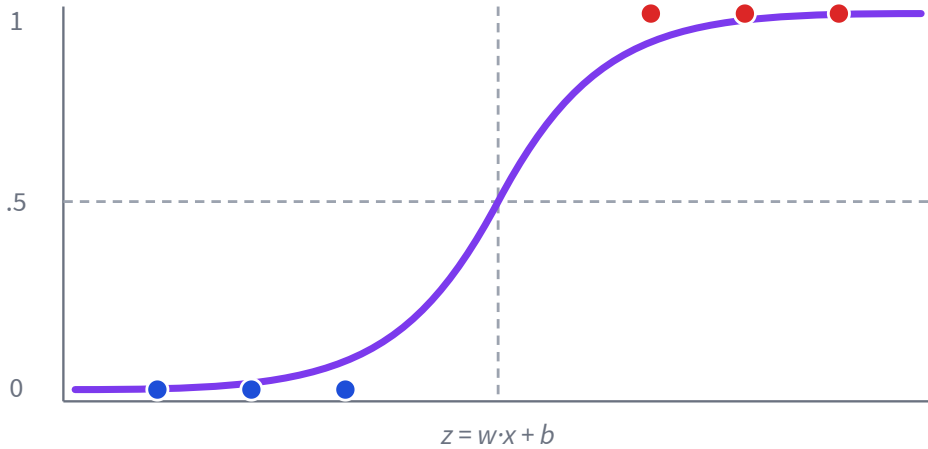
Gradient descent

Works for *any* differentiable model.
Repeat:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} L$$

Gradient descent is the workhorse for the rest of the module — the models just get bigger.

Classification: logistic regression



Squash the linear score into a **probability** with the sigmoid:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = \mathbf{w}^\top \mathbf{x} + b$$

- $\hat{y} = P(y=1 \mid \mathbf{x})$.
- Predict class 1 when $\hat{y} > 0.5$, i.e. $z > 0$.
- The boundary $z = 0$ is a **hyperplane**.

Loss for classification: cross-entropy

Squared error is a poor fit for probabilities. Instead, maximize the probability of the true labels — equivalently, minimize **binary cross-entropy**:

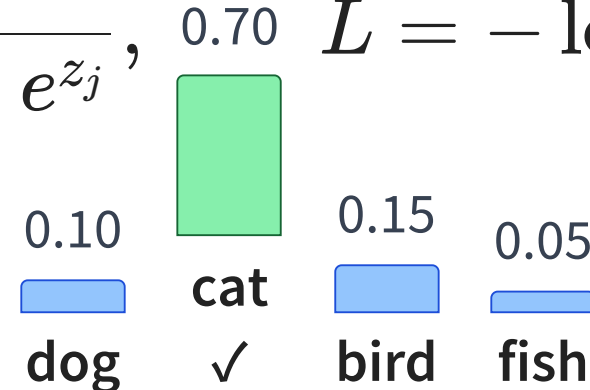
$$L(\mathbf{w}) = -\frac{1}{m} \sum_{i=1}^m \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

True label $y = 1$: loss is $-\log \hat{y}$ — small when $\hat{y} \rightarrow 1$, **huge** when $\hat{y} \rightarrow 0$.

Confident *and* wrong is punished the hardest — exactly the behaviour we want.

Many classes: softmax + cross-entropy

Output one score z_k per class; **softmax** turns scores into a probability distribution:

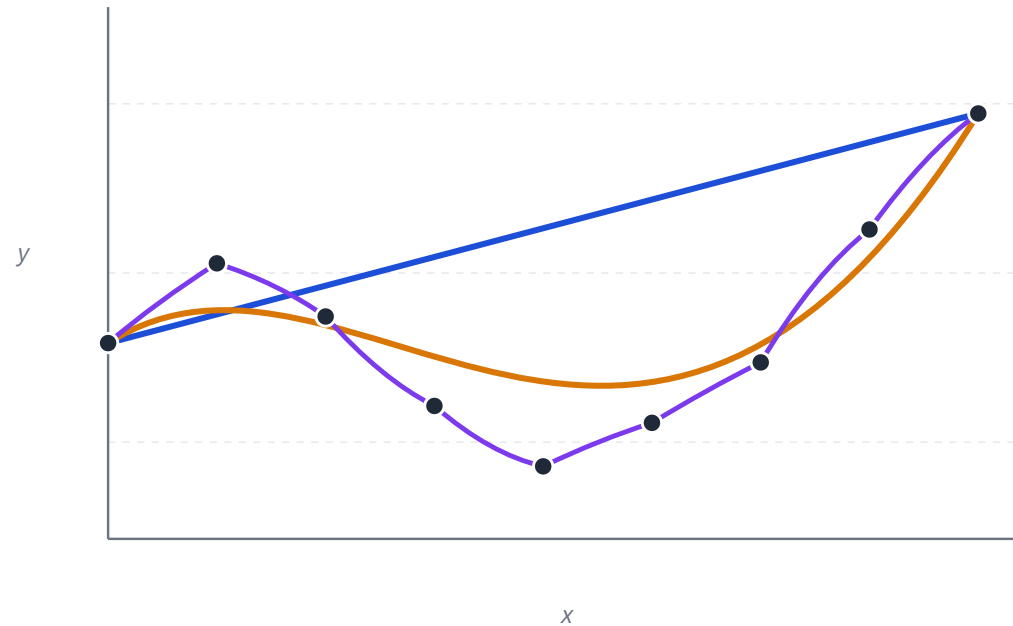
$$P(y=k \mid \mathbf{x}) = \frac{e^{z_k}}{\sum_j e^{z_j}}, \quad L = -\log P(y=\text{true class})$$


Class	Probability
dog	0.10
cat	0.70
bird	0.15
fish	0.05

This softmax + cross-entropy is exactly the output layer we'll reuse for neural nets (L20).

More power is not always better

Same 9 points, increasingly flexible models. The wiggliest one fits training data best — but would you trust it on a new x ?



— degree 1 (underfits) — degree 3 (good fit) — degree 9 (overfits)

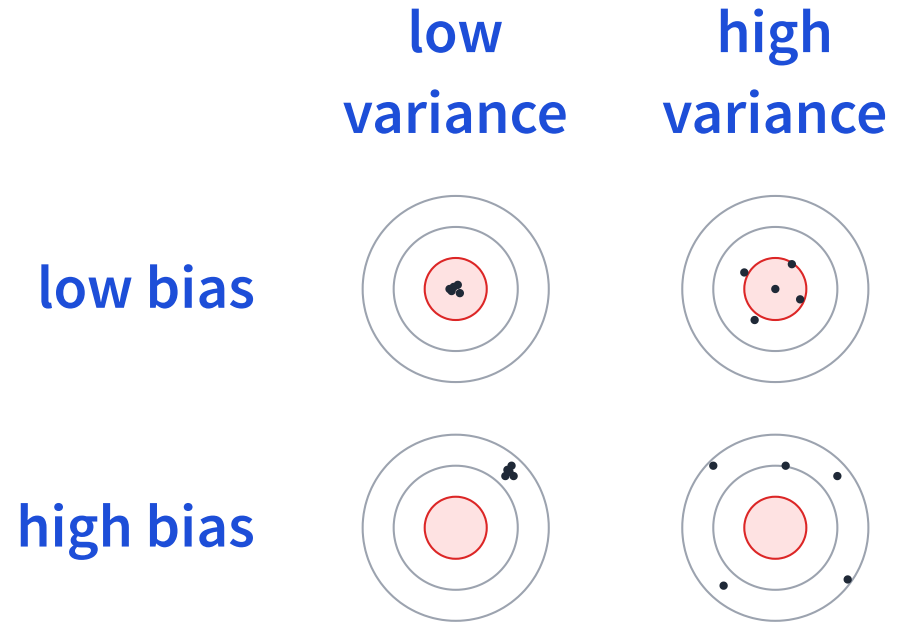
Bias and variance

Bias

Even with infinite data, how far off is the model family? High bias = too simple, **underfits**.

Variance

How much does the fit swing with a different training set? High variance = too flexible, **overfits**.



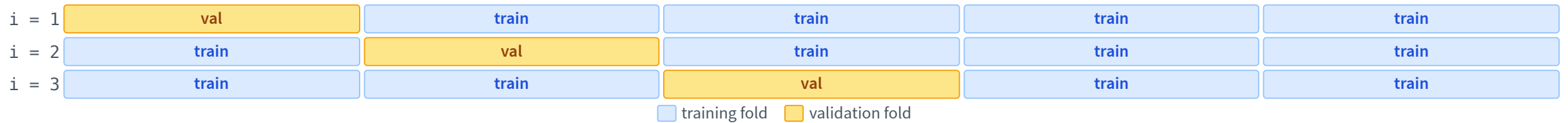
Dots = predictions from re-trained models.
Bullseye = truth.

K-fold cross-validation

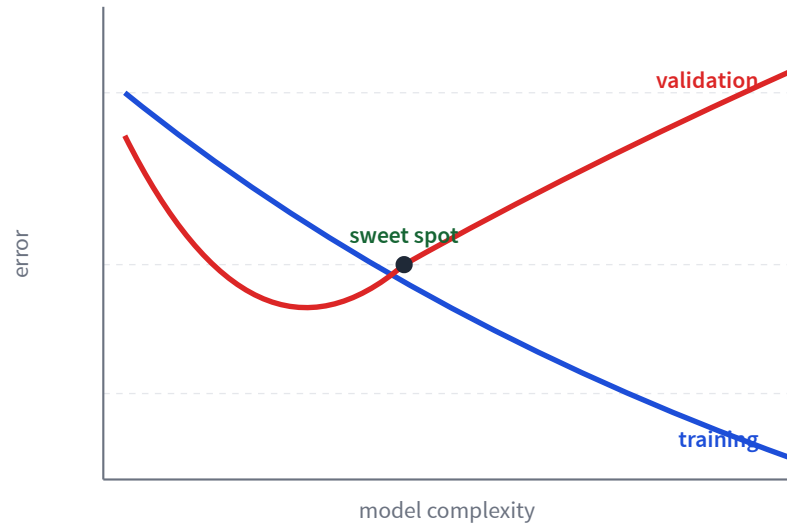
How to pick the right complexity? Use a slice of training data as a **surrogate test set**.

$k\text{-fold-CV}(\text{model}, \text{dataset}, K)$

1. Split training data into K equally sized folds.
2. For $i = 1, \dots, K$: train on the other $K - 1$ folds; evaluate on fold i .
3. Average the error over the K runs; keep the hyperparameters with the best average.



Diagnosing & fighting overfitting



Training error keeps dropping;
validation error turns back up —
that gap is overfitting.

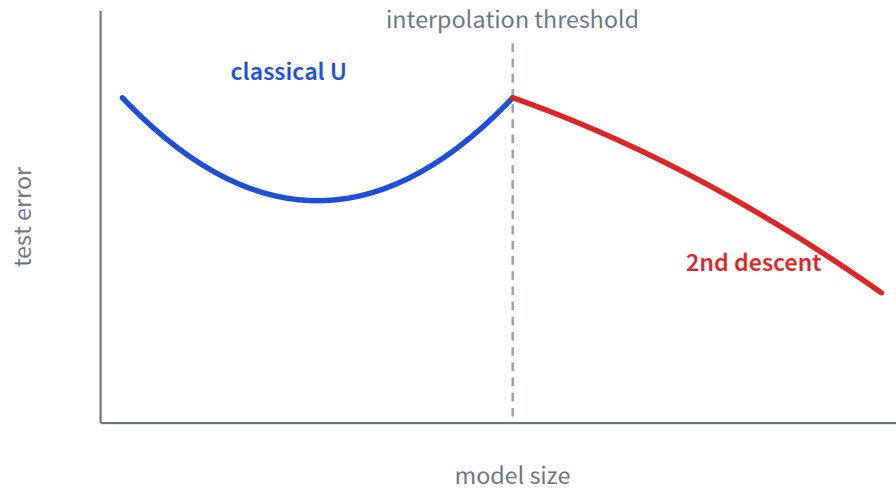
Regularization

Add a penalty on large weights
to the loss:

$$L(\mathbf{w}) + \lambda \|\mathbf{w}\|^2$$

λ trades bias against variance;
tune it with cross-validation.

A modern twist: double descent



The classical U-curve isn't the whole story.

- Past the point where the model fits training data *exactly*, test error can **fall again**.
- Very large models (deep nets, LLMs) often generalize *better*, not worse.
- Why? Still an active research question — "bigger" changed the rules.

Learning goals (recap)

- ✓ Name the types of learning; classification vs regression.
- ✓ Supervised learning = **model + loss + fit**.
- ✓ **Linear** and **logistic regression**; least squares & gradient descent.
- ✓ **Cross-entropy** and **softmax** for classification.
- ✓ Control overfitting with **cross-validation** and **regularization**.

Next: unsupervised & representation learning

Today, labels guided every hypothesis. Without labels, what *structure* can we still find in data?

L18: clustering (k-means), PCA, autoencoders, and learned representations.